CS430/910: Foundations of Data Analytics

Time Series | Ayşe Sunar

Introduction and Outline

In pretty much every scientific field, measurements are performed over time.

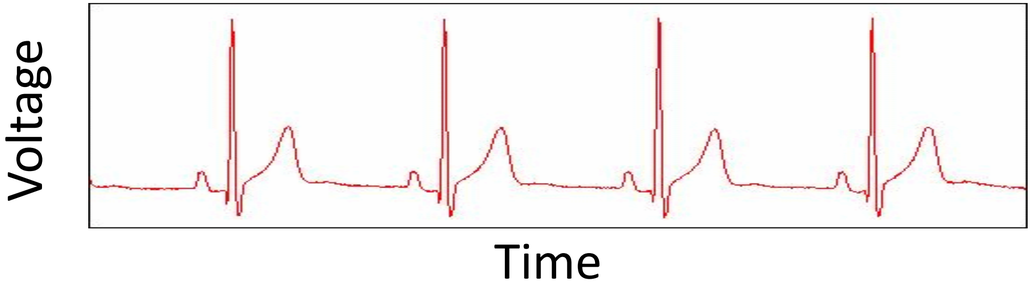
The purpose of analysing data over time series is to extract all the meaningful knowledge from the *shape* of the data.

This is a complex problem for computers. We are going to look at:

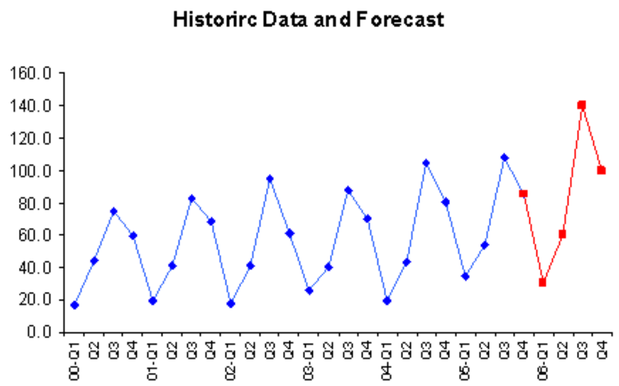
- Definition of time series and key applications
- Comparing time series: **dynamic time warping**
- Reducing the size of time series: **dimensionality reduction**
- Predicting time series: **autoregressive moving averages**
- Identifying change: **cumulative sum (CUSUM)**

Time Series

- A sequence of (numeric) values, read at (regular) intervals
  - $T = (t_1, t_2, \ldots, t_n)$ with each t_i a real number
  - E.g. Temperature read every minute
  - Usually single values but could be multiple values at each step
- Drawn from many possible domains
  - Finance: stock values, other commodities
  - Sensors: temperature, pressure, rainfall, light…
  - Networks: traffic amount, velocity...
- Want to model, compare and predict time-series data



Time Series vs Sequences



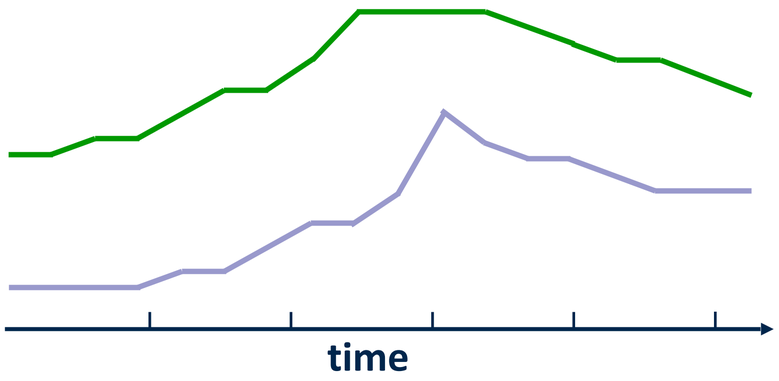
- Examples of time series:
  - Stock price over the course of a day
  - Temperature measured at noon each day for a month
- Examples of sequences:
  - Sequence of nucleotides on a chromosome (AGTTGCATA…)
  - The sequence of exam scores, sorted by student id
- Time series **must** have a time component
  - All time series are sequences, but not all sequences are time-series
  - Time series must have an explicit notion of time in them

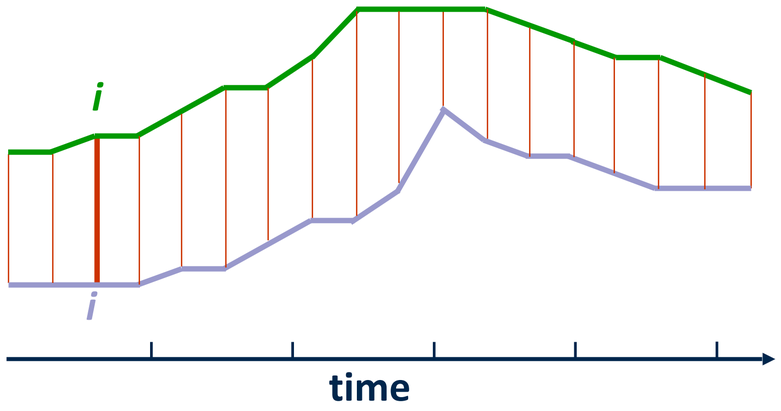
Time Series Analysis

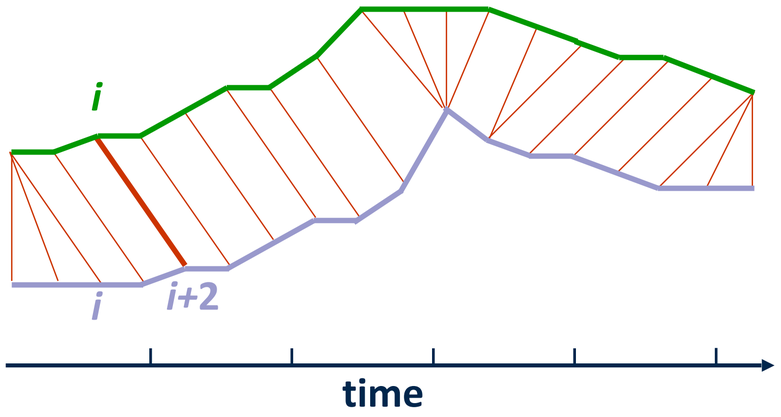
- Working with time series poses many challenges:
  - Dealing with high volumes of data
  - Respecting the special nature of the time dimension
- Will study some key problems in time series analysis:
  1. Defining similarity between pairs of time series
  2. Creating compact representations of time series
  3. Predicting values in time series
  4. Identifying changes (anomalies) in time series

Task 1: Time Series Similarity

- How to measure similarity between two time series?







Euclidean/Manhattan distance point-by-point may do poorly

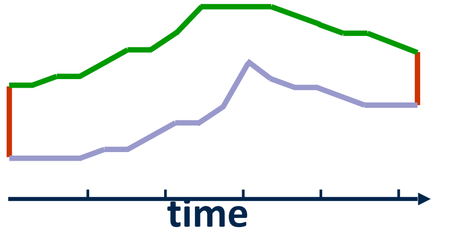
Can we map points from one series to another to get a better match?

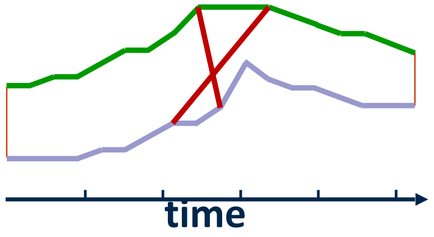
Time Series Alignment

Find a mapping between two series t (length n), u (length m)

Mapping is a series of pairs $p_1 \ldots p_L : (n_1, m_1) \ldots (n_L, m_L)$ so that:

1. First points are mapped together, last points are mapped together: $p_1 = (1,1), p_L = (n, m)$
2. Monotonicity: no "crossovers" in the mapping, so $n_1 \leq n_2 \leq \ldots \leq n_L$ and $m_1 \leq m_2 \leq \ldots \leq m_L$
3. No big steps: $p_{l+1} - p_l$ is in $\{(1,0), (0,1), (1,1)\}$





Note: 3 implies 2

Mapping Cost

The ``efficiency" of a mapping is measured by its cost.

The **cost of a mapping** given by path p is $c_p(t, u) = \sum_{j=1}^{L} c(t_{n_j}, u_{m_j})$

$c(\,.\,,\,.\,)$ is a local cost function e.g. absolute difference of values.

Define the distance between t and u as min $\{c_p(t, u) \,|\, p$ is valid$\}$

Call this the "**warping distance**", written as $D(t, u)$.

Properties of Warping Distance

- May be several different paths that achieve same minimum cost.
- Distance is symmetric, i.e., $D(t, u) = D(u, t)$, provided function c is symmetric.
- Distance may be zero even if $t \neq u$

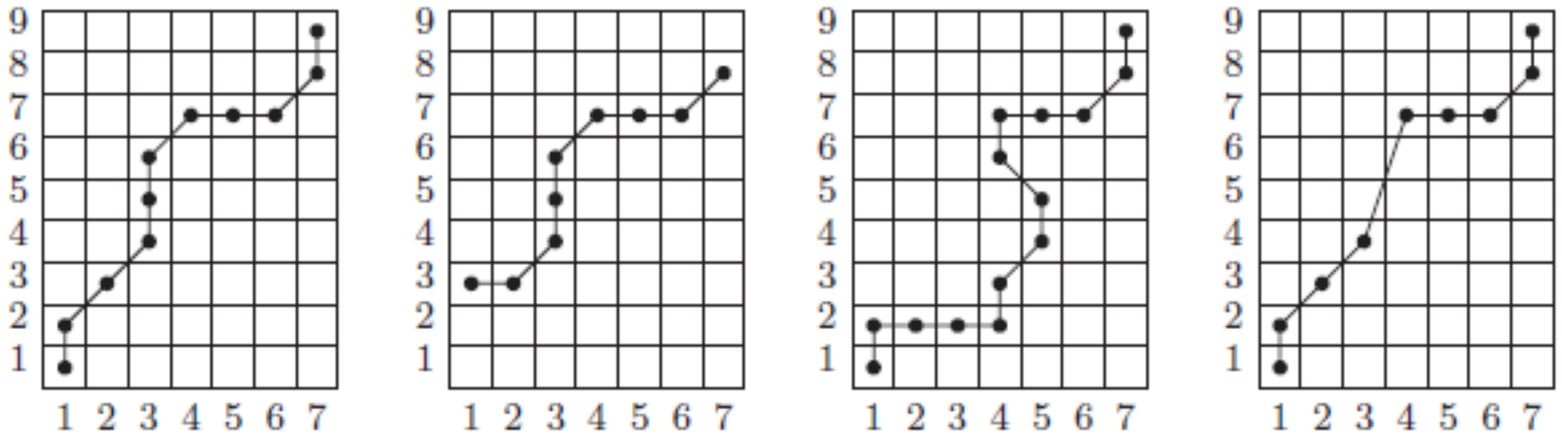
<u>Example</u>:

$t = (0,1,2), u = (0,1,1,2), v = (0,2,2)$ and $c =$ Hamming Distance

$D(t, u) = 0, D(t, v) = 1, D(u, v) = 2$

$D(u, v) > D(t, u) + D(t, v)$: does not satisfy triangle inequality

Permissible Paths



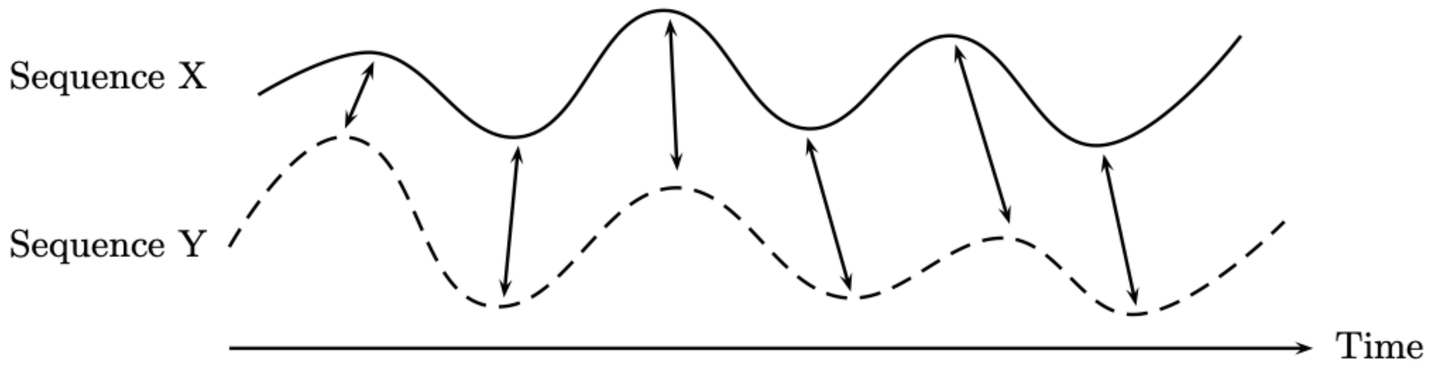
Violates condition 1 (start/end)

Violates condition 2 (monotonicity)

Violates condition 3 (big steps)

- Paths can be drawn as steps through a grid
  - Conditions on permissible paths limit the possible routes
  - Can find optimal paths via **dynamic programming (DP)**

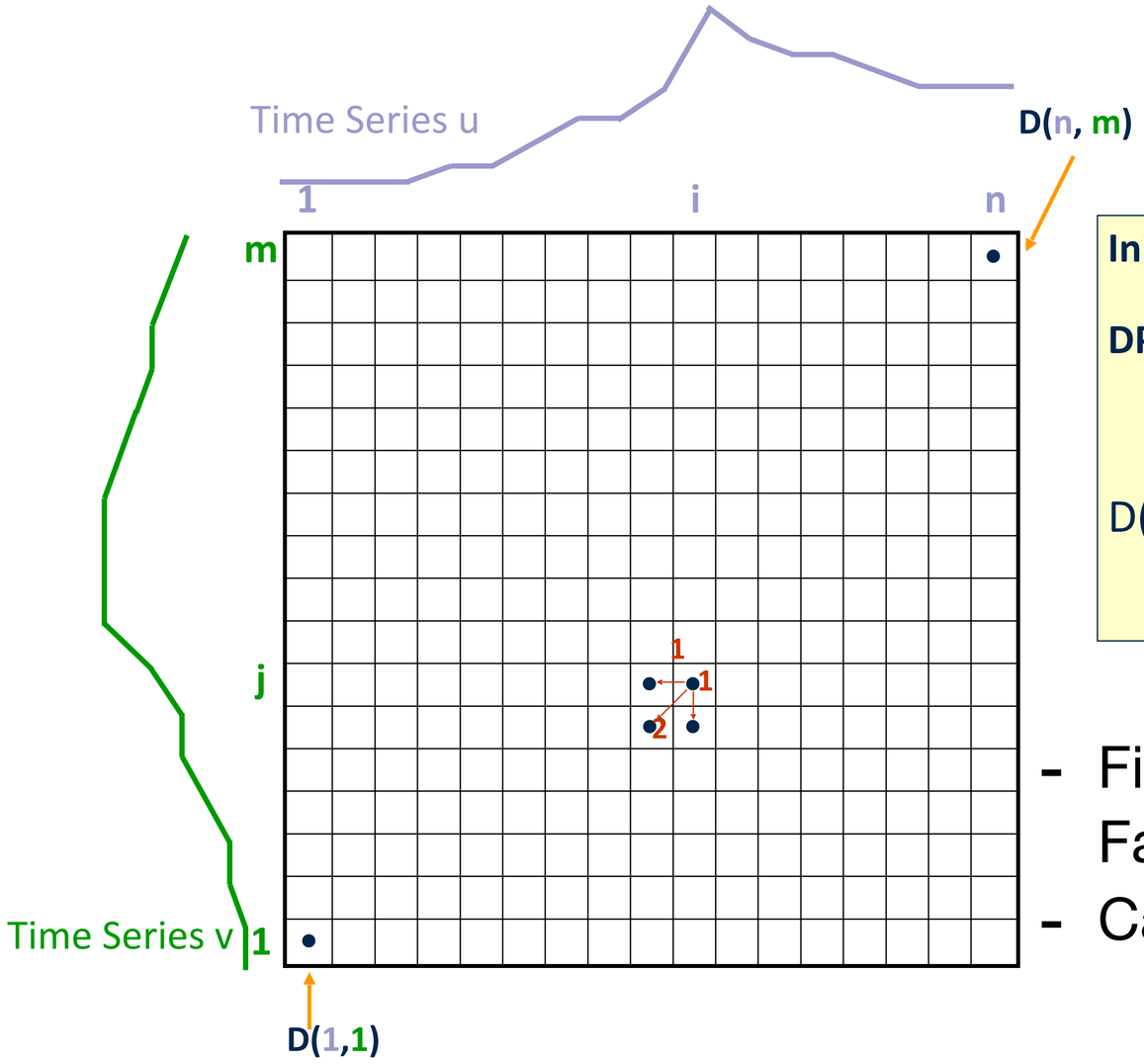
Dynamic Time Warping



Dynamic time warping (DTW) is a technique to find an optimal alignment between two given (time-dependent) sequences under certain restrictions.

Successfully applied to automatically cope with time deformations and different speeds associated with time-dependent data.

Dynamic Time Warping



Initial condition: D(1,1) = c(1,1).

DP-equation:

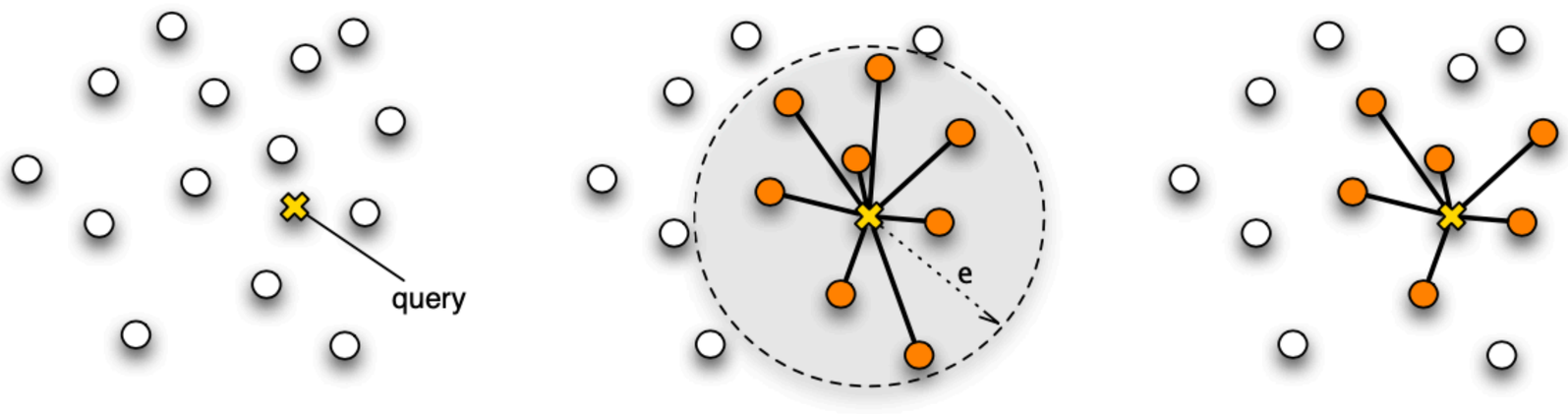
$$D(i, j) = \min \begin{pmatrix} D(i, j-1) + c(i, j) \\ D(i-1, j-1) + c(i, j) \\ D(i-1, j) + c(i, j) \end{pmatrix}$$

- Fill in matrix starting from (1,1)
  Fairly efficient: $O(nm)$ time
- Can recover path from matrix

Dynamic Time Warping Variations

- Limit how large a “shift” is allowed between sequences
  - Require $|i - j| < r$ for some bound r
  - Equivalent to only looking at a diagonal band in the matrix
  - Improves time cost to $O(r(n + m))$
- Have different step size restrictions instead of $(1,0), (0,1), (1,1)$
  - Avoid long “horizontal/vertical” subpaths
- Speed up computation by approximation
  - Coarsen sequences: average k values together
- Related to (Levenshtein) edit distance between text strings, i.e., minimum number of single character edits needed for matching with correct word
  - Similar DP, but each symbol matches at most one in the other sequence

Task 2: Dimensionality Reduction



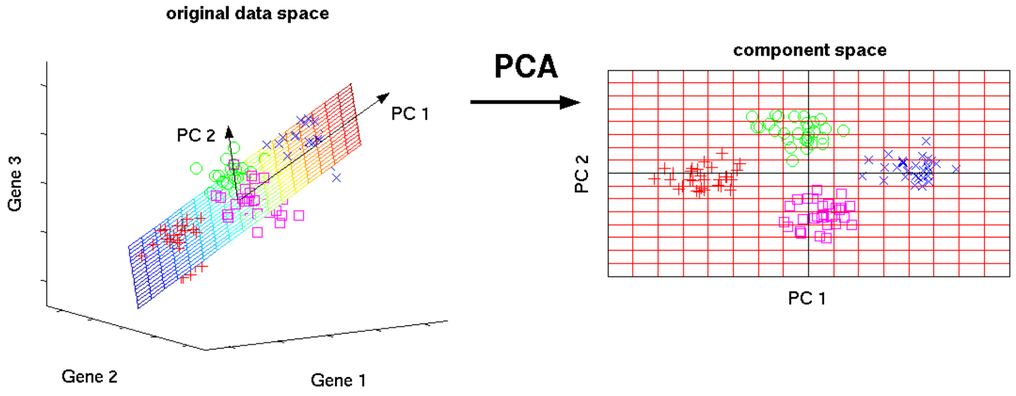
When a query is entered into the system, it is first transformed into the same representation as that used for other datapoints.

Above, an ε–*range query and* a *K–Nearest Neighbours query*

Given a time series $T = (t_1, \ldots, t_n)$ of length n, a representation of T is a model T' of reduced dimensionality d' $(d' \ll n)$ such that T' closely approximates T .

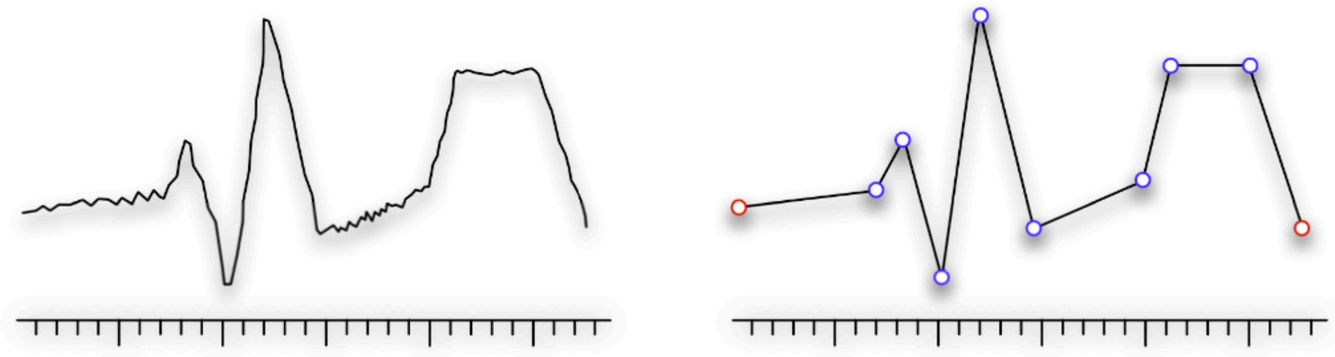
Task 2: Dimensionality Reduction

- Some time series tasks involve many different time series
  - Clustering, classification, searching
  - E.g. applied to time series of prices for all shares
  - Each time series can grow without limit
- With many large time series, analysis is slow and cumbersome
  - Reduce to an approximate representation: **dimensionality reduction**
- E.g. use approximate representation to reject candidate matches
  - Then compute similarity based for smaller pool of possibilities



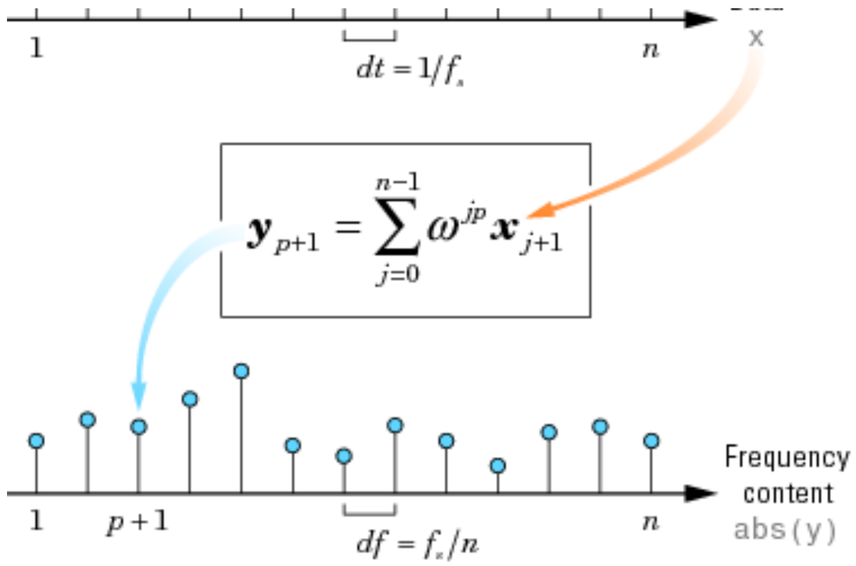
Segmentation

The segmentation (or *summarisation*) task aims at creating an accurate approximation of time series, by reducing its dimensionality while retaining its essential feature.



From usually noisy time series containing a very large number of datapoints, the goal is to find the closest approximation of the input time series with the maximal dimensionality reduction factor without loosing its essential features.

Data Transformations

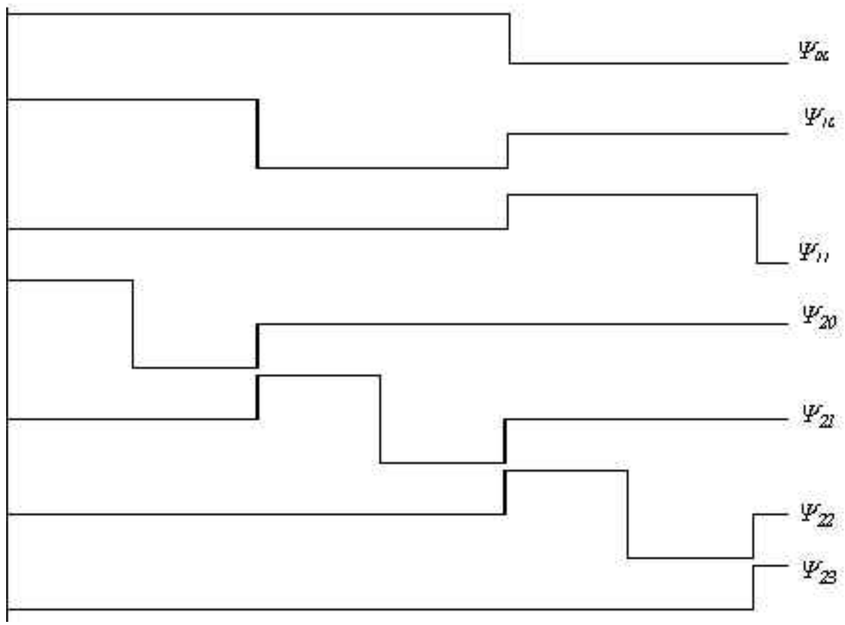


- Apply a mathematical **transformation** to data
  - Represents the same data in a different way
- Drop (zero out) some of the coefficients of the transformation
  - Losing detail, but in less important parts of the data
  - Use the remaining coefficients as the reduced representation
- Example: (discrete) Fourier transformation
  - Moves from the time domain to the frequency domain
  - If data is represented by a few periodic components, little loss
  - Usually pick the first few Fourier coefficients to represent data
- Fourier transformation can be computed quickly
  - $O(n \log n)$ for time series of n observations

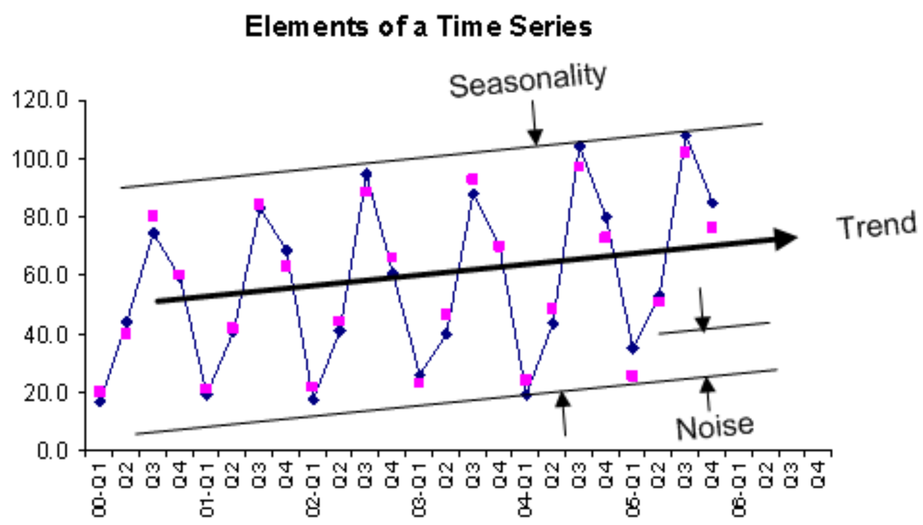
Other Transformations

- The (Haar) wavelet transformation is even faster
  - Expresses the data vector as a linear combination of wavelets
  - Each wavelet has a location and a width
  - Can be computed hierarchically in time $O(n)$

- Many other wavelet transformations (Daubechies, DCT…)
- Other techniques (mentioned elsewhere): SVD, PCA



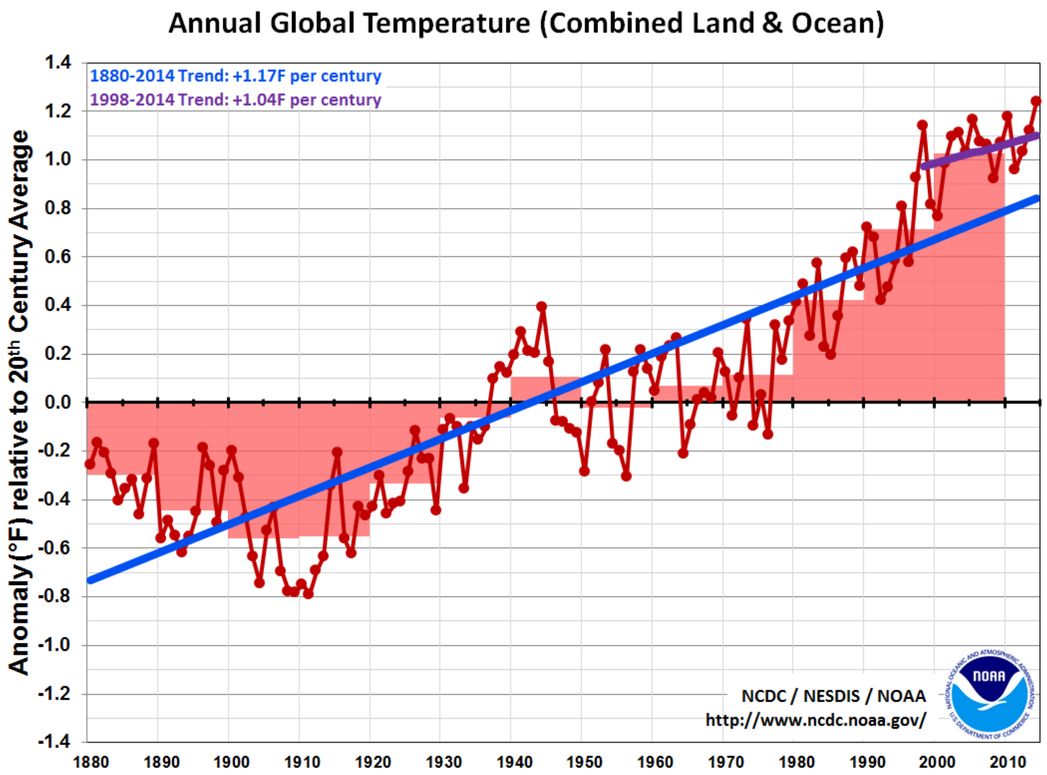
Task 3: Modelling Time Series



Time series have different kinds of structure:

- Trends: broad changes as time increases
  - E.g. prices tend to increase over time
  - Can model as a linear trend, or more complex (polynomial) function
- Seasonality: behaviour with a fixed period
  - E.g. temperatures have a seasonality with a period of a year
  - Can be multiple seasons: both daily and weekly seasons
- Cyclicity: repeating behaviour without a fixed period
  - E.g. Economic cycles (boom/bust, growth/recession)
- Irregular movements: unpredictable local variations (noise)
  - Mean that we can’t predict the future with perfect accuracy!

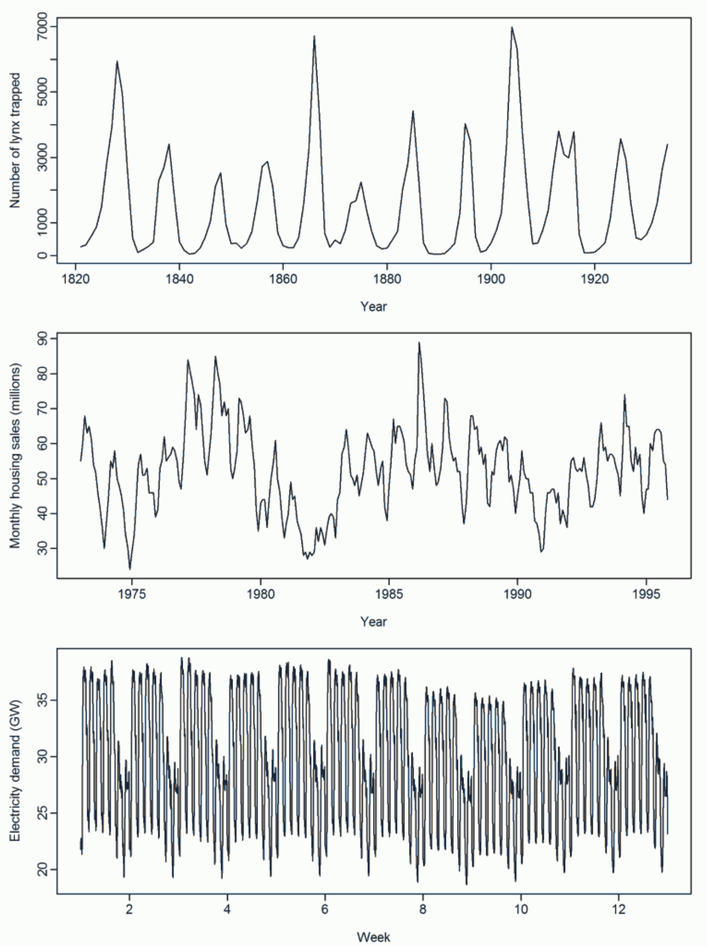
Trends in Time Series



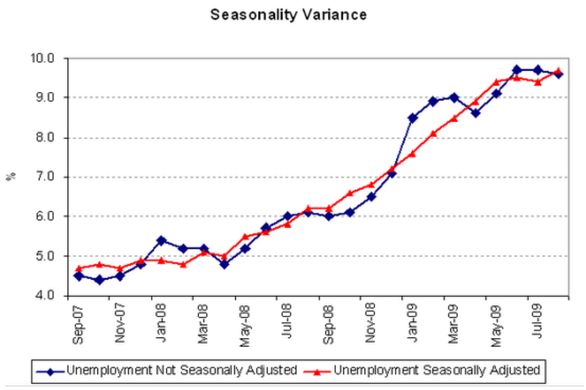
- Temperatures have annual seasonality (not shown in this plot)
- Simple linear trend fitted to temperature data
- Clear upward trend during 20th Century

Cyclicity and Seasonality

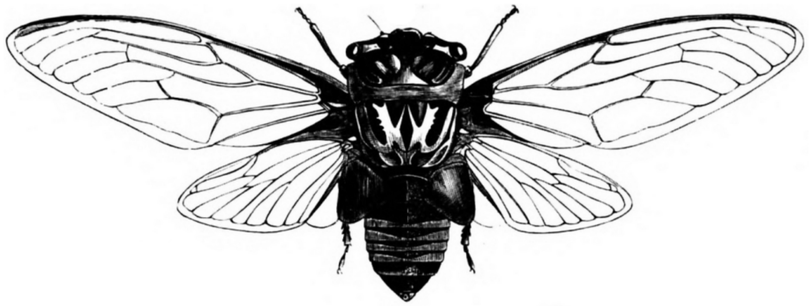
- Top plot: number of lynx captured each year in McKenzie river
  - Cyclic, approximately 8-10 years
- Middle plot: monthly sales of new homes in USA
  - Some cyclicity, approx 6-10 years
  - Strong annual seasonality
- Bottom plot: half-hourly electricity demand in England and Wales
  - Strong weekly and daily seasonality
  - Likely to be also annual seasonality



Identifying Seasonality



- Sometimes seasonality is obvious
  - A plot of the data shows daily/annual cycles
- In other cases, the seasonality is less obvious
  - "Periodic cicadas" emerge only every 17 years…
- Test for seasonality: correlate series with a shifted copy
  - Extract pairs (t_i, t_{i+k}) for a lag parameter k
  - Test if the collection of pairs has high correlation
  - Report seasonal with period k if correlation is high enough



Removing Seasonality

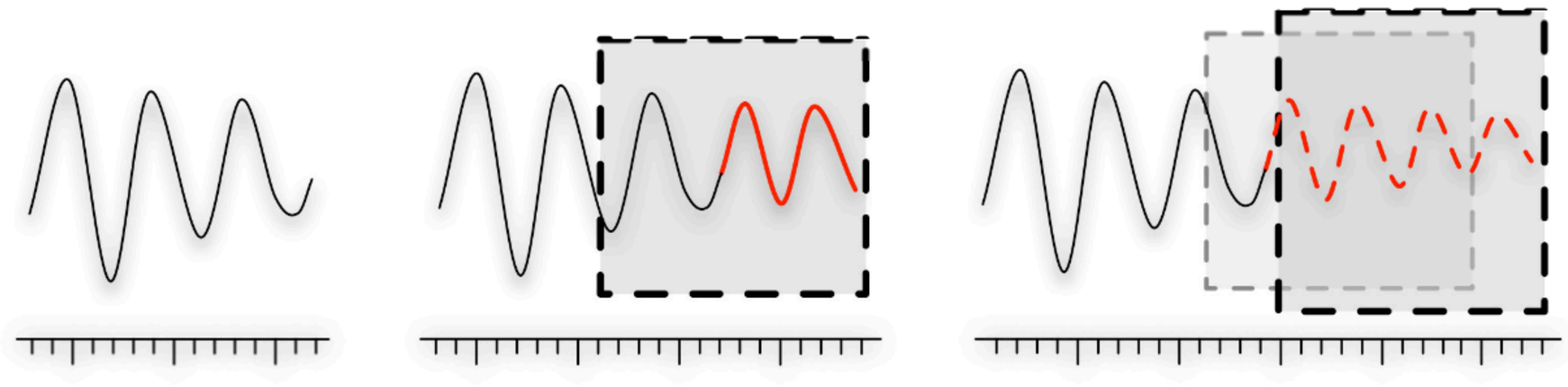
- If we know the period of the seasonality, we can try to remove it
- A simple approach is based on repeated averaging
  - Take the average of corresponding points (e.g. of all June readings)
  - Find the ratio of each seasonal average to the global average
  - Divide each data point by its seasonal ratio
- Govt. agencies (National Statistics Office) use seasonal adjustment
  - X-12-ARIMA developed by US Census is commonly used

		Raw Data			Quarter	Seasonal			Adjusted	
Quarter	2010	2011	2012		Average	Index		2010	2011	2012
1	47	43	44		44.67	0.9069		51.82	47.41	48.51
2	49	55	43		49.00	0.9949		49.25	55.28	43.22
3	53	58	58		56.33	1.1438		46.34	50.71	50.71
4	44	48	49		47.00	0.9543		46.11	50.30	51.35
					49.25	1.0000				

Time Series Modelling

- Goal: come up with a model to describe time series values
  - Incorporate seasonality, trends, and cycles if needed
- Give a formula to predict next value based on history
  - Don't expect to be exactly right
  - Aim to minimize error, e.g. RMSE error
- In principle, the next value could depend on entire history
  - Hard to model (too many parameters), so simplify
  - Markov assumption: next value depends on only recent values
    - E.g. Only need to look at last two values to predict next

Prediction



Can we *predict* time series? A typical prediction task:

(a) The input time series may exhibit a periodical and thus predictable structure.

(b) The goal is to forecast a maximum number of upcoming datapoints within a prediction window.

(c) The task becomes really hard when it comes to having recursive prediction, i.e. the long term prediction of a time series implies reusing the earlier forecast values as inputs in order to go on predicting.

Simple Recurrent Predictors

- Simplest possible model: predict next value is same as last
  - $p_{i+1} = t_i$
  - "If it rained today, it will rain tomorrow"
  - Baseline to compare against
- Can make generalizations
  - Incorporate (linear) trend: $p_{i+1} = t_i + (t_i - t_{i-1})$
    Assume constant difference between successive observations
  - Rate of change: $p_{i+1} = t_i \times (\frac{t_i}{t_{i-1}})$
    Assume constant ratio between successive observations

Averaging Methods

The last value may be quite changeable. Use averaging to remove variability

Simple approach: always predict the global average value: works well if time series is stationary (no trend, cycles, period).

- Moving average: compute average of last k observations

  For suitable k: 5? 10? 100? 1000?

  predict $p_i = \sum_{j=1}^{k} t_{i-j}/k$

  Easy to update: add on new term, subtract k'td

- Exponential Moving Average: decay contribution of old values

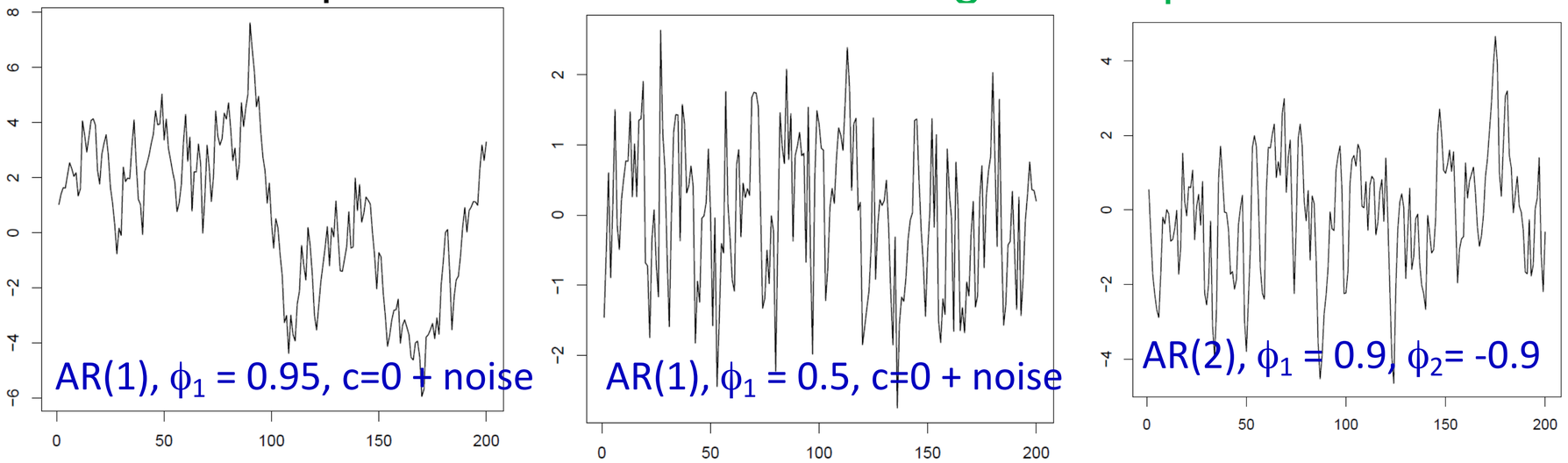
  Predict $p_i = \sum_{j=1}^{n} \alpha(1-\alpha)^j t_{i-j}$ for smoothing parameter $0 < \alpha < 1$

  Easy to update: next prediction $p_i = \alpha t_{i-1} + (1-\alpha)p_{i-1}$

Autoregressive Moving Average Models

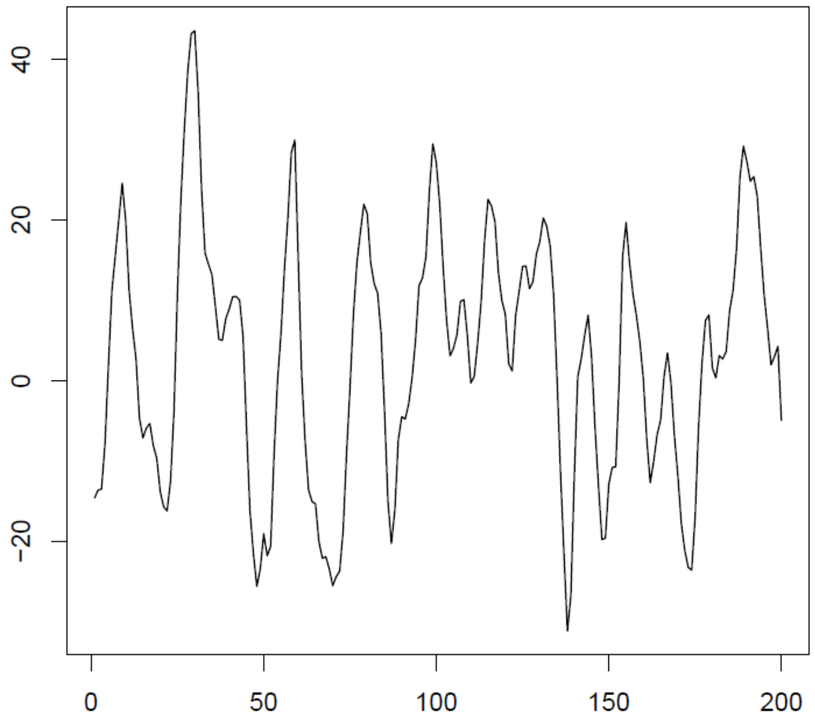
- Learn the parameters of how t_i depends on recent values
  - Autoregressive moving average (ARMA) models
- Consist of two parts
  - Autoregressive model: predicts based on k recent values
  - Moving average: predict based on recent prediction errors
  - Prediction is the sum of two parts
- To simulate model, may additionally add “noise” to the output

Autoregressive (AR) Model

- Assume that value depends on k recent values (seasonality, trend)
- $p_i = c + \sum_{j=1}^{k} \phi_i t_{i-j}$: with parameters $c, \varphi_1, \varphi_2, \ldots \varphi_k$
- How to learn the model parameters (assume k fixed)?
- Train on historic data, aim to minimize error $(t_i - p_i)$
- Minimize squared error: a multilinear regression problem
- 

ARMA Models

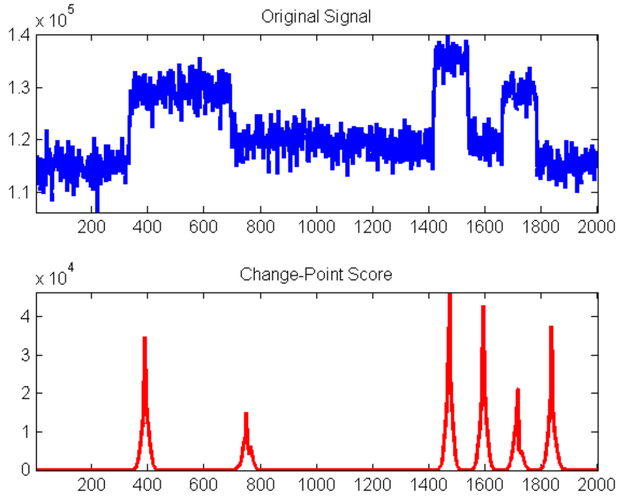
ARMA(2,4)



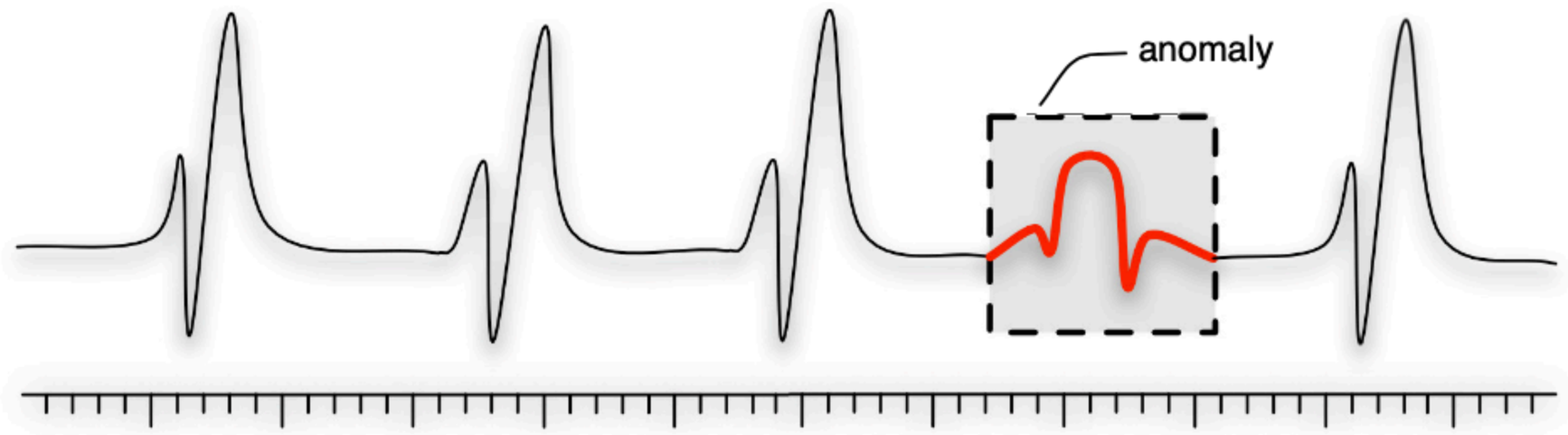
To predict m steps ahead: predict 1 step ahead, then the next...

Task 4: Time Series Change Detection

- We want to detect when there has been a change in the data
  - E.g. to alert if there is an emergency (sudden rise in temperature)
- Several aspects to trade-off:
  - False-alarm rate: a random variation triggers an alarm
  - Misdetection rate: we fail to detect a real change
  - Detection Delay: how long after a change before it is detected
- Often we want to do this online: as new readings are made



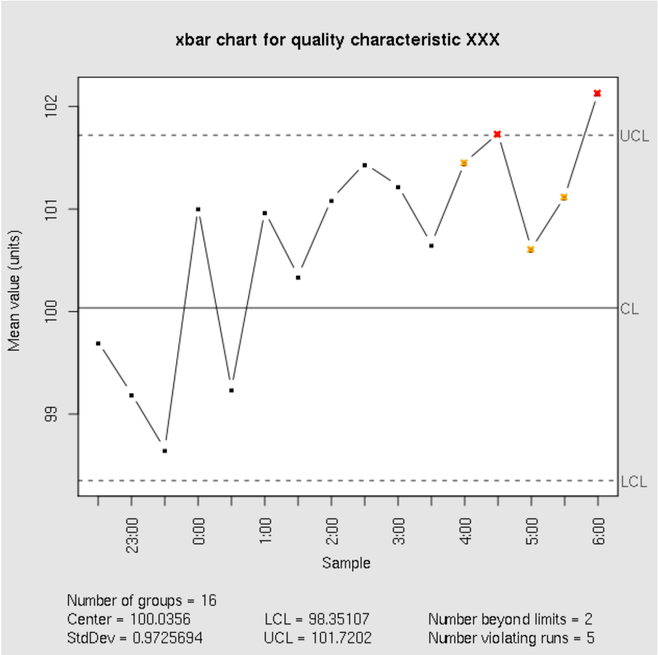
Anomaly Detection



An idealized example of the anomaly detection task. A long time series which exhibits some kind of periodical structure can be modeled thanks to a reduced pattern of “standard” behaviour. The goal is thus to find subsequences which does not follow the model and may therefore be considered as anomalies.

Simple Change Detection

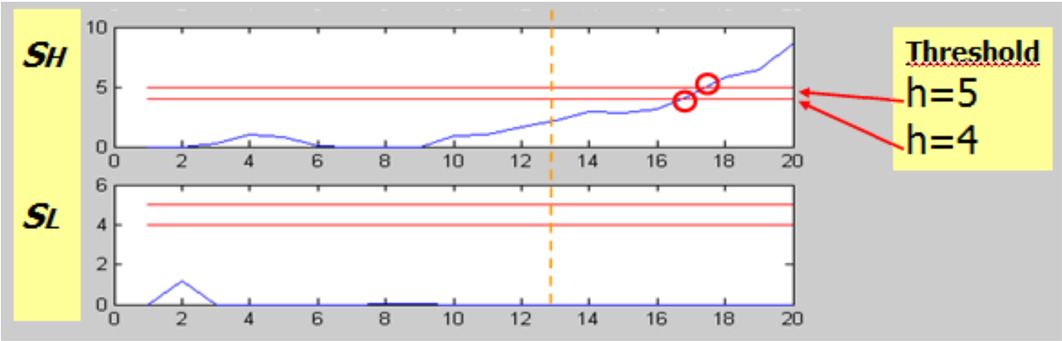
- Easy if the data is very stable (normal around a mean value)
  - Keep track of mean, standard deviation
  - Alert if the values are more than 3 SDs away from the mean
- May be captured on a control chart
  - Plot data, mean, and control lines
  - Upper and lower controls, based on acceptable deviation from mean
- Can use a moving average to smooth out local variations (spikes)
  - And remove seasonality etc.



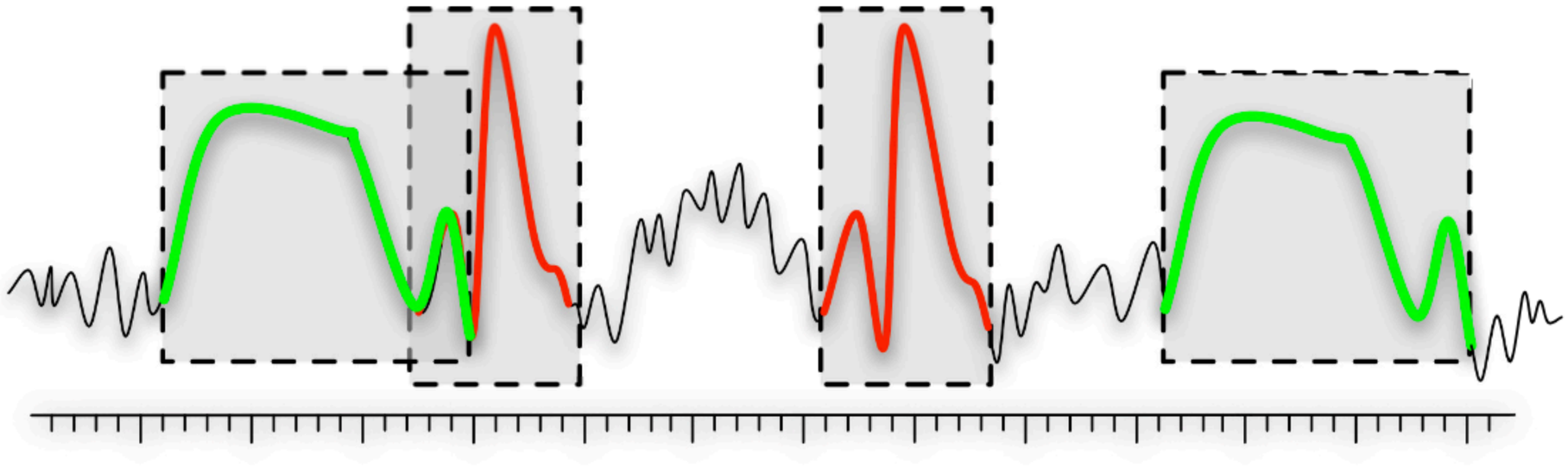
CUSUM: Cumulative Sum

- CUSUM aims to detect changes earlier
  - Keeps a cumulative sum statistic
  - When this statistic exceeds a threshold, alert
- Given observation t_i and expected value E, compute
  - $SH \leftarrow SH + max(t_i - E, 0)$: sum of higher than expected values
  - $SL \leftarrow SL + max(E - t_i, 0)$: sum of lower than expected values
  - Alert if S_H or S_L exceeds some (tunable) threshold h

    Choice of h trades off false alarms and misdetection
  - Identifies when there is a small shift in the mean without spikes



Motif Detection



The task of motif discovery consists in finding every subsequence that appears recurrently in a longer time series.

These subsequences are named *motifs*.

This task exhibits a high combinatorial complexity as several motifs can exist within a single series, motifs can be of various lengths and even overlap.

Summary and Further Reading

- Comparing time series: dynamic time warping
- Reducing the size of time series: dimensionality reduction
- Predicting time series: autoregressive moving averages
- Identifying change: cumulative sum (CUSUM)

Background Reading

- Survey: “Time series data mining”, P. Esling and C. Agon
  http://repmus.ircam.fr/_media/esling/ts_review_short.pdf
- Dynamic Time Warping M. Muller (book chapter in “Information Retrieval for Music and Motion)